



Cyberscope

A *TAC Security* Company

Audit Report

Phoenix Token

Aug 2026

Repository <https://github.com/phoenixtoken-pxt/phoenix-protocol/>
Commit 752cf676042f43adc31eed8b2a6e047bb6307ec1
Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	4
Review	5
Audit Updates	5
Source Files	5
Overview	7
Pxt.sol	7
PxtFeeModel.sol	7
PxtSellAccess.sol	7
PhoenixFeeCollector.sol	7
PhoenixBuyback.sol	8
PhoenixBuybackMath.sol	8
PhoenixV4ReturnDeltaHook.sol	8
Findings Breakdown	9
Diagnostics	10
PBRD - Persistent Band Registry DoS	12
Description	12
Recommendation	12
ST - Stops Transactions	13
Description	13
Recommendation	15
APTE - Alternate Pool Tax Evasion	16
Description	16
Recommendation	18
CMBS - Collector Misaccounts Buyback Solvency	19
Description	19
Recommendation	19
FBBP - Flash-Borrowed Balance Bypasses Penalty	21
Description	21
Recommendation	22
JLVE - JIT Liquidity Value Extraction	23
Description	23
Recommendation	23
LRSA - Liquidity Refund Sweeps Accruals	24
Description	24
Recommendation	24
PTTB - PoolManager Transfer Tax Bypass	25
Description	25
Recommendation	25

RIFM - Requested Input Fee Mispricing	26
Description	26
Recommendation	28
RAAAU - Router Allowlist Authorizes All Users	29
Description	29
Recommendation	29
UBFS - Unspent Buyback Funds Stranded	30
Description	30
Recommendation	30
CCIB - Code-Length Check Is Bypassable	31
Description	31
Recommendation	31
CCR - Contract Centralization Risk	32
Description	32
Recommendation	33
EOIPF - Expected Output Ignores Pool Fees	34
Description	34
Recommendation	35
EORZ - Expected Output Rounds Zero	36
Description	36
Recommendation	37
FWEPE - Fixed Windows Enable Penalty Evasion	38
Description	38
Recommendation	38
FAPR - Fresh Address Penalty Reset	40
Description	40
Recommendation	40
IDUT - Incorrect Default Unlock Timestamp	41
Description	41
Recommendation	41
LFRU - LP Fees Remain Unallocated	42
Description	42
Recommendation	42
MC - Missing Check	43
Description	43
Recommendation	44
OPIIU - Official Pool Initialization Is Unrestricted	45
Description	45
Recommendation	46
RCRP - Rotated Contracts Retain Privileges	47
Description	47
Recommendation	48

STCBT - Seed Transfers Can Be Taxed	49
Description	49
Recommendation	50
L04 - Conformance to Solidity Naming Conventions	51
Description	51
Recommendation	51
Functions Analysis	52
Summary	58
Disclaimer	59
About Cyberscope	60

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit

30 Jul 2026

Source Files

Filename	SHA256
src/uniswap/v4/V4Addresses.sol	140f35c3f855b9cc91e7982d32e06a1d4b4c6ce84103a033ef42a11a34c4521d
src/uniswap/v4/HookMiner.sol	45821cd13e0fc55bf1902cdb2d6b8b2f47fe5e17529fb008f15229042004e22b
src/return-delta/PhoenixV4ReturnDeltaHook.sol	73611c46abb899cf3cf1a4a5665cbc0a79888d9095f978014ead54cc5eec3aab
src/return-delta/ISellAttributor.sol	372eab49ca0392b7cbba2af3cf4bdafa87781ef9f311f44d91ef3cc74e9080f1
src/fee/PhoenixFeeCollector.sol	90ebe3869eb952e199112afa33b74ab553e6419b75c2d46f3df1a2d669a4727d
src/fee/PhoenixBuybackMath.sol	887acd9a305c4dcf14593e35c86feb4ae302a744b5a6a8fb11305de859e98f31
src/fee/PhoenixBuyback.sol	4454438260235ca706a982e0786dfd040f9658ddb5f6f3b13cddd13fa7ffdae5
src/core/PxtSellAccess.sol	ccad99715e3c49fd57fdbe201275035c2fd98559714c91f70b988b9a9f914da8
src/core/PxtFeeModel.sol	3eeacd01b9544db0a03d26a8086e40cbfb46cd57fcbdfc853ffc29c949d5262d

src/core/Pxt.sol

f2845ad5c2705f95dafb2f4cc9ed92d688a
32c1a65cfaabe7f979af47a5476ae

Overview

Phoenix Token is a Base-focused token ecosystem built around PXT, a fixed-supply, six-decimal ERC-20 integrated with a Uniswap v4 ReturnDelta hook. The protocol applies transfer, buy, sell, and dump-penalty fees; enforces a timed sell lock and anti-bot opening ceremony; restricts transfers to unapproved contracts; collects fees in USDC; burns part of sold PXT; and uses accumulated buyback fees to purchase PXT and recycle it into single-sided liquidity positions.

Pxt.sol

`Pxt` implements the 971-billion-supply Phoenix Token with ERC-20 Permit support. It charges a 2.7% wallet-transfer fee, enforces the sell lock and anti-bot controls, manages special wallet statuses and approved contract recipients, burns sell fees, and reports authenticated DEX settlements to the configured sell-attribution hook.

PxtFeeModel.sol

`PxtFeeModel` defines the protocol's fee types, constants, errors, and pure calculation functions. It divides buy and transfer fees between donation and marketing wallets, calculates the fixed PXT burn on sells, and splits normal or penalty USDC sell fees among donation, marketing, and buyback obligations.

PxtSellAccess.sol

`PxtSellAccess` provides shared checks for the immutable sell-unlock timestamp and anti-bot state. It distinguishes between seller-specific ERC-20 settlement checks and the general "trading open" condition used by the hook and fee collector.

PhoenixFeeCollector.sol

`PhoenixFeeCollector` is the protocol treasury that receives and accounts for fees skimmed by the hook. Its permissionless `collect` operation pulls Uniswap LP fees, pays donation and marketing obligations, burns available PXT, reconciles underfunded balances, and preserves the USDC buyback allocation for later execution.

PhoenixBuyback.sol

`PhoenixBuyback` supplies the fee collector's liquidity and buyback machinery. It owns the protocol's seed position, swaps accrued USDC for PXT subject to deadline and slippage controls, deposits purchased PXT into bounded single-sided recycle bands, collects position fees, and allows empty recycle ranges to be pruned.

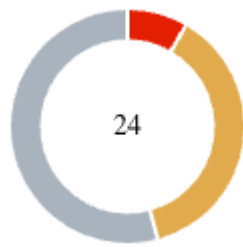
PhoenixBuybackMath.sol

`PhoenixBuybackMath` contains the pure arithmetic used by the buyback system. It converts quote-token value into expected PXT output, derives price limits, enforces minimum output, aligns ticks to pool spacing, and constructs and validates PXT-only recycle ranges.

PhoenixV4ReturnDeltaHook.sol

`PhoenixV4ReturnDeltaHook` is the Uniswap v4 hook for the official PXT/USDC pool. It charges fees through return deltas, skims USDC on buys and sells, coordinates PXT burns, applies the rolling 24-hour dump penalty, rebates eligible sellers after token-authenticated attribution, handles orphaned ERC-6909 settlements, gates liquidity during the sell lock, and forwards finalized fees to the collector.

Findings Breakdown



● Critical	2
● Medium	9
● Minor / Informative	13

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	2	0	0	0
● Medium	9	0	0	0
● Minor / Informative	13	0	0	0

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	PBRD	Persistent Band Registry DoS	Unresolved
●	ST	Stops Transactions	Unresolved
●	APTE	Alternate Pool Tax Evasion	Unresolved
●	CMBS	Collector Misaccounts Buyback Solvency	Unresolved
●	FBBP	Flash-Borrowed Balance Bypasses Penalty	Unresolved
●	JLVE	JIT Liquidity Value Extraction	Unresolved
●	LRSA	Liquidity Refund Sweeps Accruals	Unresolved
●	PTTB	PoolManager Transfer Tax Bypass	Unresolved
●	RIFM	Requested Input Fee Mispricing	Unresolved
●	RAAAU	Router Allowlist Authorizes All Users	Unresolved
●	UBFS	Unspent Buyback Funds Stranded	Unresolved
●	CCIB	Code-Length Check Is Bypassable	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	EOIPF	Expected Output Ignores Pool Fees	Unresolved

●	EORZ	Expected Output Rounds Zero	Unresolved
●	FWEPE	Fixed Windows Enable Penalty Evasion	Unresolved
●	FAPR	Fresh Address Penalty Reset	Unresolved
●	IDUT	Incorrect Default Unlock Timestamp	Unresolved
●	LFRU	LP Fees Remain Unallocated	Unresolved
●	MC	Missing Check	Unresolved
●	OPIIU	Official Pool Initialization Is Unrestricted	Unresolved
●	RCRP	Rotated Contracts Retain Privileges	Unresolved
●	STCBT	Seed Transfers Can Be Taxed	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved

PBRD - Persistent Band Registry DoS

Criticality	Critical
Location	PhoenixBuyback.sol#L230,305,449,504
Status	Unresolved

Description

Recycle liquidity cannot be removed, while pruning deletes only positions whose liquidity is already zero. Permissionless callers can create 32 distinct nonzero bands and move the price until none remains PXT-only. Once the registry is full,

`_selectPxtOnlyRecycleBand` reverts and future buybacks remain unavailable.

```
if (!recycleBandRegistered[idealId]
    && recycleBands.length >= MAX_RECYCLE_BANDS) {
    (lo, hi) = _selectPxtOnlyRecycleBand(tick, pxtIsToken0);
}

revert RecycleBandNotPxtOnly();
```

Recommendation

Consider supporting controlled position migration, removal and replacement of unusable bands, or a recycling design with bounded and recoverable state.

ST - Stops Transactions

Criticality	Critical
Location	Pxt.sol#L123,215,260 PxtSellAccess.sol#L21 PhoenixV4ReturnDeltaHook.sol#L182,304
Status	Unresolved

Description

While ownership remains active, the owner can call `setAntiBotSeller()` after the sell-unlock timestamp and reset `sellProtectionCleared` to `false`. Subsequent ERC-20 sells execute `_enforceSellAccess()` before `_onDexSell()`, so only the newly designated anti-bot seller can pass and clear the protection. If the owner selects an address that cannot sell—or repeatedly resets the flag—public sells can be blocked indefinitely.

Hook-mediated sells are also rejected by `enforceTradingOpen()` while the flag is false, whereas buys and ordinary transfers follow separate paths and remain operational. This gives the owner a selective sell-freezing capability after trading has already opened. The documented ownership-renunciation ceremony removes this capability only if it is completed successfully.

```
function setAntiBotSeller(address seller) external onlyOwner {
    if (seller == address(0)) revert ZeroAddress();
    antiBotSeller = seller;
    sellProtectionCleared = false;
}

_enforceSellAccess(from);
super._update(from, to, amount);
_onDexSell(from, amount);
```

Trading can additionally be halted through the protocol's mandatory external dependencies. Every ERC-20 sale settled to the PoolManager makes a synchronous call to `sellAttributor.attributeSell()`, and every fee finalization makes a further synchronous call to `feeCollector.receiveAccruedFees()`. Neither call is wrapped in a failure-tolerant pattern, so if the configured contract reverts, is paused, is incompatible with the expected interface, or consumes excessive gas, the entire sale reverts.

`setSellAttributor()` and `setFeeCollector()` accept any nonzero address without verifying that the target implements the expected interface or behaves correctly. An incorrect or subsequently broken configuration therefore disables selling for all users while leaving buys and transfers operational — reproducing the same selective sell-freeze described above, but without requiring any further owner action once the address is set.

The dependency also propagates beyond individual sales. An orphaned USDC skim is finalized automatically at the beginning of the next swap through `__beforeSwap()`, which calls `finalizeOrphanedSell()` and in turn `__accrueUsdc()`. If the collector call cannot succeed, that single unfinalizable orphan reverts every subsequent official-pool swap, extending the failure from sells alone to all pool activity until the collector is replaced.

```
if (address(sellAttributor) != address(0)) {
    sellAttributor.attributeSell(seller, amount);
}

IERC20(token).forceApprove(address(feeCollector), total);
feeCollector.receiveAccruedFees(token, kind, donation, marketing, 0,
buyback);

if (orphanSkim.usdcSkim > 0) {
    finalizeOrphanedSell();
}
```

Recommendation

Consider making the sell-protection lifecycle irreversible after the unlock timestamp or after it has first been cleared. Anti-bot seller configuration could be restricted to the pre-unlock initialization phase, and subsequent calls should not be able to restore an already-cleared sell restriction.

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

APTE - Alternate Pool Tax Evasion

Criticality	Medium
Location	Pxt.sol#L219 PhoenixV4ReturnDeltaHook.sol#L182,217
Status	Unresolved

Description

PXT identifies DEX settlement solely by transfers to or from the singleton Uniswap v4 PoolManager and exempts these transfers from the 2.7% token transfer fee. The actual buy, sell, burn, and dump-penalty fees are enforced only when swaps pass through the official Phoenix hook.

An attacker can create a separate PXT/USDC pool on the same PoolManager with no hook. Transfers for this pool still receive privileged settlement treatment, while the Phoenix hook never executes. On sells, `attributeSell()` finds no pending official swap and returns silently. Consequently, both the standard 5.4% sell fee and the 37.8% dump penalty can be bypassed entirely.

```
if (poolManager != address(0) && to == poolManager && from !=
poolManager) {
    super._update(from, to, amount);
    _onDexSell(from, amount);
}

uint256 pendingPxt = _tload(_T_PXT_IN);
if (pendingPxt == 0) return;

...

function attributeSell(address seller, uint256 pxtAmount) external {
    if (msg.sender != address(pxt)) revert OnlyPxt();
    uint256 pendingPxt = _tload(_T_PXT_IN);
    // LP / non-swap settlements also hit PoolManager; ignore when no
    pending swap skim.
    if (pendingPxt == 0) return;
    if (pxtAmount != pendingPxt) revert PendingSellMismatch();

    uint256 usdcOut = _tload(_T_USDC_OUT);
    uint256 skimmed = _tload(_T_USDC_SKIM);
    address quoteAddr = address(uint160(_tload(_T_QUOTE)));

    _clearPendingSkim();

    // balanceAfter settlement + sold amount = pre-sell balance.
    uint256 balanceBefore = pxt.balanceOf(seller) + pxtAmount;
    uint256 fairBps = _fairSellFeeBps(seller, pxtAmount, balanceBefore);
    _applySellWindow(seller, pxtAmount, balanceBefore);

    uint256 refund;
    if (fairBps == 0) {
        refund = skimmed;
    } else if (fairBps == PxtFeeModel.SELL_FEE_BPS && usdcOut > 0) {
        (uint256 dPen, uint256 mPen, uint256 bPen) =
PxtFeeModel.splitSellUsdc(PxtFeeModel.PENALTY_FEE_BPS, usdcOut);
        (uint256 dSell, uint256 mSell, uint256 bSell) =
PxtFeeModel.splitSellUsdc(PxtFeeModel.SELL_FEE_BPS, usdcOut);
        uint256 fairSkim = dSell + mSell + bSell;
        uint256 penSkim = dPen + mPen + bPen;
        if (penSkim > fairSkim) refund = penSkim - fairSkim;
        if (refund > skimmed) refund = skimmed;
    }

    if (refund > 0 && quoteAddr != address(0)) {
        IERC20(quoteAddr).safeTransfer(seller, refund);
    }

    uint256 keep = skimmed - refund;
    if (keep > 0 && quoteAddr != address(0)) {
```

```
uint256 feeBps = fairBps == 0 ? PxtFeeModel.SELL_FEE_BPS :
fairBps;
(uint256 donation, uint256 marketing, uint256 buyback) =
PxtFeeModel.splitSellUsdc(feeBps, usdcOut);
uint256 target = donation + marketing + buyback;
// Rounding: distribute `keep` (may be 1 wei off vs target).
if (target > 0 && keep != target) {
    donation = (keep * donation) / target;
    marketing = (keep * marketing) / target;
    buyback = keep - donation - marketing;
} else if (target == 0) {
    buyback = keep;
}
FeeKind kind = fairBps == PxtFeeModel.PENALTY_FEE_BPS ?
FeeKind.Penalty : FeeKind.Sell;
_accrueUsdc(Currency.wrap(quoteAddr), kind, donation, marketing,
buyback);
}

emit SellAttributed(seller, pxtAmount, fairBps, refund);
}
```

Recommendation

Bind privileged token settlement to authenticated activity in the official pool.

CMBS - Collector Misaccounts Buyback Solvency

Criticality	Medium
Location	PhoenixFeeCollector.sol#L81,141 PhoenixBuyback.sol#L191,284
Status	Unresolved

Description

The collector records donation, marketing, burn, and buyback obligations against a shared token balance, but `_payoutToken()` and `_reconcileWalletBurnPending()` account for only the first three. Buyback funds are not reserved before wallet payments or burns, and `pendingBuyback` is excluded when outstanding obligations are reconciled against the available balance.

If the collector becomes undercollateralized—such as when `addLiquidity()` returns all residual balances to the owner—remaining assets may be paid to wallets while the buyback obligation stays unchanged. `_executeBuybackCash()` later limits spending to the smaller real balance but reduces `pendingBuyback` only by the amount actually spent, leaving a phantom pending balance that future reconciliation never removes.

```
uint256 totalDue = donationDue + marketingDue + burnDue;

uint256 due = don + mkt + burn;

uint256 left0 = IERC20(token0).balanceOf(address(this));
uint256 left1 = IERC20(token1).balanceOf(address(this));
if (left0 > 0) IERC20(token0).safeTransfer(msg.sender, left0);
if (left1 > 0) IERC20(token1).safeTransfer(msg.sender, left1);

uint256 reserved = pendingDonation[quote] + pendingMarketing[quote];
uint256 spendable = bal > reserved ? bal - reserved : 0;
pendingBuyback[quote] = usdcBudget - spend;
```

Recommendation

Consider including every pending obligation in a unified solvency calculation and reserving buyback backing before other balances are spent. Reconciliation should adjust all liability

categories when assets are insufficient, while liquidity operations should distinguish newly supplied seed funds from pre-existing balances backing accrued obligations.

FBBP - Flash-Borrowed Balance Bypasses Penalty

Criticality	Medium
Location	PhoenixV4ReturnDeltaHook.sol#L217,466,486
Status	Unresolved

Description

The dump-penalty threshold uses the seller's balance immediately before their first sale as the denominator for the entire 24-hour window. Although subsequent sales are accumulated correctly, `balanceAtWindowStart` is never adjusted when the seller's balance decreases through transfers or other PoolManager settlements. The snapshot can therefore include tokens that the seller only controls temporarily.

A seller contract can transiently borrow `PXT` from the `PoolManager`, execute a small sale to initialize the window with an inflated balance, and repay the borrowed `PXT` before the unlock ends. The repayment is not added to `soldInWindow` because it has no pending swap attribution. The seller can subsequently sell their actual position while the 10% threshold is still calculated against the inflated snapshot, reducing an otherwise penalized 37.8% sale to the standard 5.4% tier. The required inflation is limited by available pool liquidity.

```
uint256 balanceBefore = pxt.balanceOf(seller) + pxtAmount;
uint256 fairBps = _fairSellFeeBps(seller, pxtAmount, balanceBefore);
_applySellWindow(seller, pxtAmount, balanceBefore);

if (window.windowStart == 0 || block.timestamp >= window.windowStart +
PxtFeeModel.PENALTY_WINDOW) {
    window.windowStart = uint64(block.timestamp);
    window.soldInWindow = 0;
    window.balanceAtWindowStart = balanceBefore;
}

bool penalized =
    balanceAtStart > 0 && newSold * PxtFeeModel.BPS > balanceAtStart *
PxtFeeModel.PENALTY_THRESHOLD_BPS;
```

Recommendation

Consider deriving the penalty denominator from a balance measure that cannot be inflated with same-transaction liquidity, such as a checkpointed historical balance or a baseline that is conservatively adjusted when the seller's holdings decrease. `PoolManager` transfers and repayments should also be incorporated into the accounting where they can alter the balance used by the penalty window.

JLVE - JIT Liquidity Value Extraction

Criticality	Medium
Location	PhoenixBuyback.sol#L264,281 PhoenixBuybackMath.sol#L17,34,45
Status	Unresolved

Description

Anyone may execute a predictable protocol buyback, and both its expected output and price boundary derive from the pool's immediately observable spot price. An attacker can manipulate spot or place concentrated JIT liquidity, trigger the buyback with a zero caller minimum, and unwind afterward, extracting value while remaining inside the protocol's spot-relative tolerance.

```
(uint160 sqrtPriceX96,,, ) = poolManager.getSlot0(id);  
  
uint160 sqrtLimit =  
    PhoenixBuybackMath.sqrtPriceLimitAfterSlippage(  
        sqrtPriceX96, zeroForOne, maxBuybackSlippageBps  
    );  
uint256 expectedPxt =  
    PhoenixBuybackMath.pxtForQuote(spend, sqrtPriceX96, pxtIsToken0);
```

Recommendation

Avoid relying solely on instantaneous pool spot for permissionless protocol trades.

LRSA - Liquidity Refund Sweeps Accruals

Criticality	Medium
Location	PhoenixBuyback.sol#L191,203
Status	Unresolved

Description

`addLiquidity` refunds the contract's complete post-operation balances instead of only the unused portion of the caller's contribution. If fee obligations have accrued before an owner liquidity addition, the call transfers their backing assets to the owner while leaving the pending accounting unchanged.

```
poolManager.unlock(abi.encode(ACTION_ADD, abi.encode(params,
hookData)));

uint256 left0 = IERC20(token0).balanceOf(address(this));
uint256 left1 = IERC20(token1).balanceOf(address(this));
if (left0 > 0) IERC20(token0).safeTransfer(msg.sender, left0);
if (left1 > 0) IERC20(token1).safeTransfer(msg.sender, left1);
```

Recommendation

Snapshot balances before accepting the caller's contribution and limit refunds to assets attributable to the current operation. Protocol obligations should remain reserved throughout liquidity-management operations.

PTTB - PoolManager Transfer Tax Bypass

Criticality	Medium
Location	Pxt.sol#L206,241,274 PhoenixV4ReturnDeltaHook.sol#L217
Status	Unresolved

Description

PXT treats every transfer into or out of PoolManager as privileged settlement. A holder can deposit PXT, transfer the corresponding ERC-6909 claims without invoking PXT, and redeem them to another address. This bypasses the ordinary 2.7% transfer fee and permits delivery to an otherwise unapproved contract.

```
if (poolManager != address(0) && from == poolManager) return;

if (poolManager != address(0) && (from == poolManager || to ==
poolManager)) {
    return PxtFeeModel.noFee(amount);
}
```

Recommendation

Authenticate the specific swap or liquidity operation associated with each PoolManager transfer.

RIFM - Requested Input Fee Mispricing

Criticality	Medium
Location	PhoenixV4ReturnDeltaHook.sol#L304,341,373,422
Status	Unresolved

Description

Fee calculations use inconsistent bases across swap modes. For exact-input buys and sell burns, the hook calculates charges before execution from the requested input. If a price limit causes a partial fill, the charge still reflects the full request and can materially overcharge the executed trade. The mirror issue occurs for exact-output swaps: fees are calculated as a percentage of the requested net output or raw swap amount and then added on top. For example, requesting 100 USDC during a penalized sell causes the pool to provide 135.95 USDC in total, of which 35.95 USDC is retained. The effective fee is therefore only 26.44% of gross output, rather than 35.95%. Exact-output sell burns similarly represent approximately 1.816% of the trader's total PXT payment instead of 1.85%, while exact-output buys exhibit the same undercharging pattern.

```
function _beforeSell(PoolKey calldata key, SwapParams calldata params)
    internal
    returns (bytes4, BeforeSwapDelta, uint24)
{
    // Time unlock + anti-bot cleared: blocks ERC-6909 before
    clearSellProtection.
    PxtSellAccess.enforceTradingOpen(IPxtSellControls(address(pxt)));

    // Exact-in: burn fixed 1.85% of PXT (same for sell and penalty).
    if (_pxtIsSpecified(key, params)) {
        uint256 pxtAmount = SignedMath.abs(params.amountSpecified);
        if (pxtAmount == 0) revert ZeroAmount();
        uint256 burnAmount =
PxtFeeModel.splitSellBurn(PxtFeeModel.SELL_FEE_BPS, pxtAmount);
        if (burnAmount == 0) {
            return (BaseHook.beforeSwap.selector, toBeforeSwapDelta(0,
0), 0);
        }
        poolManager.take(pxtCurrency, address(this), burnAmount);
        pxt.burnBalance(burnAmount);
        return (BaseHook.beforeSwap.selector,
toBeforeSwapDelta(burnAmount.toInt128(), 0), 0);
    }

    // Exact-out: pessimistic USDC skim from specified output at penalty
    rates.
    uint256 usdcAmount = SignedMath.abs(params.amountSpecified);
    if (usdcAmount == 0) revert ZeroAmount();
    (uint256 donation, uint256 marketing, uint256 buyback) =
PxtFeeModel.splitSellUsdc(PxtFeeModel.PENALTY_FEE_BPS,
usdcAmount);
    uint256 usdcFee = donation + marketing + buyback;
    if (usdcFee == 0) {
        return (BaseHook.beforeSwap.selector, toBeforeSwapDelta(0, 0),
0);
    }
    Currency quote = _quoteCurrency(key);
    poolManager.take(quote, address(this), usdcFee);
    _tstore(_T_USDC_OUT, usdcAmount);
    _tstore(_T_USDC_SKIM, usdcFee);
    _tstore(_T_QUOTE, uint256(uint160(Currency.unwrap(quote))));
    return (BaseHook.beforeSwap.selector,
toBeforeSwapDelta(usdcFee.toInt128(), 0), 0);
}

function _beforeBuy(address sender, PoolKey calldata key, SwapParams
calldata params)
    internal
    returns (bytes4, BeforeSwapDelta, uint24)
{

```

```
// Protocol buyback (FeeCollector is PoolManager.swap sender) - do
not re-tax recycle USDC.
if (address(feeCollector) != address(0) && sender ==
address(feeCollector)) {
    return (BaseHook.beforeSwap.selector, toBeforeSwapDelta(0, 0),
0);
}

if (_pxtIsSpecified(key, params)) {
    return (BaseHook.beforeSwap.selector, toBeforeSwapDelta(0, 0),
0);
}

uint256 usdcAmount = SignedMath.abs(params.amountSpecified);
if (usdcAmount == 0) revert ZeroAmount();

(uint256 donation, uint256 marketing) =
PxtFeeModel.splitBuy(usdcAmount);
uint256 usdcFee = donation + marketing;
if (usdcFee == 0) {
    return (BaseHook.beforeSwap.selector, toBeforeSwapDelta(0, 0),
0);
}

Currency quote = _quoteCurrency(key);
poolManager.take(quote, address(this), usdcFee);
_accrueUsdc(quote, FeeKind.Buy, donation, marketing, 0);

emit HookFeeCharged(address(0), FeeKind.Buy, PxtFeeModel.BUY_FEE_BPS,
usdcFee);
emit FeeCharged(address(0), address(poolManager), FeeKind.Buy,
PxtFeeModel.BUY_FEE_BPS, usdcFee);

return (BaseHook.beforeSwap.selector,
toBeforeSwapDelta(usdcFee.toInt128(), 0), 0);
}
```

Recommendation

Calculate input-denominated charges from actual swap consumption after execution. For exact-output swaps, define whether configured rates apply to gross or net amounts and apply the corresponding gross-up calculation consistently across buys, sells, burns, and penalty fees.

RAAAU - Router Allowlist Authorizes All Users

Criticality	Medium
Location	PhoenixV4ReturnDeltaHook.sol#L169
Status	Unresolved

Description

The pre-unlock liquidity gate authorizes the `sender` supplied by the PoolManager. This value is the direct caller of `PoolManager.modifyLiquidity`, which is typically a shared position manager or router rather than the beneficial liquidity provider. Allowlisting a shared router therefore authorizes every user who can access it, while allowlisting an EOA does not authorize that EOA when it operates through an unapproved router.

Consequently, users may add liquidity before the unlock through an allowlisted shared router, undermining the intended provider restriction. The unnamed `hookData` parameter is ignored, so the hook has no additional authenticated context from which to identify the actual provider.

```
function _beforeAddLiquidity(
    address sender,
    PoolKey calldata key,
    ModifyLiquidityParams calldata params,
    bytes calldata
) internal view override returns (bytes4) {
    _enforceOfficialPool(key);
    if (params.liquidityDelta > 0 && block.timestamp <
pxt.sellUnlockTimestamp()) {
        if (!liquidityProvider[sender]) revert LiquidityNotAllowed();
    }
    return this.beforeAddLiquidity.selector;
}
```

Recommendation

Consider authenticating the beneficial liquidity provider rather than relying solely on the immediate `PoolManager` caller.

UBFS - Unspent Buyback Funds Stranded

Criticality	Medium
Location	PhoenixBuyback.sol#L284,305,311,315
Status	Unresolved

Description

A price-limited exact-input swap may consume less quote token than requested. The callback settles the actual PoolManager delta but does not return it;

`_executeBuybackCash` instead reports and deducts the requested amount. This removes unspent assets from `pendingBuyback`, leaving them stranded in the collector.

```
uint256 quoteOwed = uint256(int256(-swapDelta.amount0()));  
quoteCurrency.settle(poolManager, address(this), quoteOwed, false);  
  
usdcSpent = spend;  
pendingBuyback[quote] = usdcBudget - spend;
```

Recommendation

Base accounting and minimum-output validation on the amount actually consumed by the swap.

CCIB - Code-Length Check Is Bypassable

Criticality	Minor / Informative
Location	Pxt.sol#L198,206
Status	Unresolved

Description

`_isWallet()` classifies any address with no deployed bytecode as a wallet, allowing `_enforceContractRecipient()` to bypass the contract-recipient allowlist. However, zero code does not prove that an address is permanently an EOA: an attacker can precompute a future CREATE2 address, transfer PXT to it while `code.length` is zero, and deploy a contract at that address afterward.

The newly deployed, unapproved contract retains full control over the prefunded PXT and can transfer or otherwise use it, defeating the intended restriction on sending tokens to contracts. The same underlying limitation can affect addresses during contract construction.

```
function _isWallet(address account) internal view returns (bool) {
    uint256 size = account.code.length;
    if (size == 0) return true;
    if (size != 23) return false;
    bytes memory code = account.code;
    return code[0] == 0xef && code[1] == 0x01 && code[2] == 0x00;
}

if (!_isWallet(to)) return;
if (isApprovedContractRecipient[to]) return;
revert ContractRecipientNotApproved();
```

Recommendation

The inherent inability to distinguish EOAs from undeployed contract addresses should be accounted for in the restriction's security assumptions.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	Pxt.sol#L111,123,143,151,161,167 PhoenixV4ReturnDeltaHook.sol#L112,120,126 PhoenixFeeCollector.sol#L27 PhoenixBuyback.sol#L167,180
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function setPoolManager(address poolManager_) external onlyOwner;
function setAntiBotSeller(address seller) external onlyOwner;
function setFeeCollector(address collector_) external onlyOwner;
function setSellAttributor(ISellAttributor attributor_) external
onlyOwner;
function setWalletStatus(address wallet, WalletStatus status) external
onlyOwner;

function setApprovedContractRecipient(address recipient, bool approved)
external
onlyRole(RECIPIENT_APPROVER_ROLE);

function setFeeCollector(PhoenixFeeCollector collector_) external
onlyOwner;
function setLiquidityProvider(address account, bool allowed) external
onlyOwner;
function setOfficialPool(PoolKey calldata key) external onlyOwner;

function setHook(address hook_) external onlyOwner;
function configurePool(
    PoolKey calldata key,
    int24 tickLower_,
    int24 tickUpper_,
    bytes32 salt_
) external onlyOwner;

function setBuybackParams(
    uint24 recycleWidthSpacings_,
    uint16 maxBuybackSlippageBps_
) external onlyOwner;
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

EOIPF - Expected Output Ignores Pool Fees

Criticality	Minor / Informative
Location	PhoenixBuyback.sol#L284 PhoenixBuybackMath.sol#L17,34
Status	Unresolved

Description

`_executeBuybackCash()` calculates `expectedPxt` by converting the entire USDC input at the pool's pre-swap spot price. `pxtForQuote()` does not account for the pool's LP fee, price impact, or potentially unconsumed input at the price limit. The subsequent minimum-output check therefore treats an unattainable, fee-free spot conversion as the baseline.

For example, if 100 USDC is expected to buy 10,000 PXT at spot and the configured tolerance is 2%, the protocol requires at least 9,800 PXT. A 3% pool fee produces at most 9,700 PXT even before price impact, causing an otherwise valid buyback to revert. A fee near 2% similarly leaves little or no allowance for ordinary price movement, potentially making buybacks consistently unavailable.

```
uint256 expectedPxt =
    PhoenixBuybackMath.pxtForQuote(spend, sqrtPriceX96, pxtIsToken0);

PhoenixBuybackMath.enforceMinOut(
    pxtBought,
    minPxtBought,
    expectedPxt,
    maxBuybackSlippageBps
);

uint256 protocolMin = FullMath.mulDiv(
    expected,
    PxtFeeModel.BPS - slippageBps,
    PxtFeeModel.BPS
);
if (actual < protocolMin) revert Slippage();
```

Recommendation

Consider deriving the expected output from a fee-aware pool quote that reflects the executable input and anticipated price impact. Pool fees and market slippage could be treated separately, with configuration validation ensuring that unavoidable trading costs cannot consume the entire permitted tolerance.

EORZ - Expected Output Rounds Zero

Criticality	Minor / Informative
Location	PhoenixBuybackMath.sol#L17,34
Status	Unresolved

Description

In the PXT-as-token1 branch, `pxtForQuote()` divides the squared price by `Q96` before multiplying it by `quoteAmount`. This prematurely truncates precision and can produce zero even when the mathematically combined calculation is nonzero. For example, with `sqrtPriceX96 = 2^48 - 1` and `quoteAmount = 79,228,162,514,264,900,543,497,371,652`, the implemented calculation returns zero while the full-precision result is one base unit. The token0 branch similarly introduces avoidable error through two successive rounded divisions.

When `expectedPxt` becomes zero, `enforceMinOut()` no longer applies the configured percentage floor and only requires that `actual` be nonzero. If the caller supplies a zero `callerMin`, a buyback returning a single base unit can pass, weakening the protocol's slippage protection when the spot price is at an extreme or manipulated value.

```

if (pxtIsToken0) {
    pxtAmount = FullMath.mulDiv(quoteAmount, FixedPoint96.Q96,
sqrtPriceX96);
    pxtAmount = FullMath.mulDiv(pxtAmount, FixedPoint96.Q96,
sqrtPriceX96);
} else {
    pxtAmount = FullMath.mulDiv(
        quoteAmount,
        FullMath.mulDiv(sqrtPriceX96, sqrtPriceX96, FixedPoint96.Q96),
        FixedPoint96.Q96
    );
}

if (expected == 0) {
    if (actual == 0) revert Slippage();
    return;
}

```

Recommendation

Consider calculating the conversion with a single full-precision operation or an equivalent formulation that preserves precision until the final division.

FWEPE - Fixed Windows Enable Penalty Evasion

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L466,486
Status	Unresolved

Description

The dump protection uses a fixed 24-hour bucket beginning with the seller's first sale rather than a rolling 24-hour lookback. A seller can use a dust sale to choose the window boundary, sell the remainder of their approximately 10% allowance immediately before expiry, and sell another 10% of their new starting balance immediately after the reset. This allows roughly 19% of the original position to be sold seconds apart without triggering the dump penalty.

This attack is independent of the balance-snapshot inflation variant: it requires no temporary liquidity or additional wallets. The hard reset discards all prior sales at the boundary, even when they occurred only seconds earlier, while the new sale establishes a fresh balance denominator.

```
if (
    window.windowStart == 0
    || block.timestamp >= window.windowStart +
PxtFeeModel.PENALTY_WINDOW
) {
    soldInWindow = 0;
    balanceAtStart = balanceBefore;
}

uint256 newSold = soldInWindow + amount;
bool penalized =
    balanceAtStart > 0
    && newSold * PxtFeeModel.BPS
    > balanceAtStart * PxtFeeModel.PENALTY_THRESHOLD_BPS;
```

Recommendation

Consider using a rolling 24-hour accounting mechanism that retains sales occurring within the actual lookback period instead of resetting all history at a fixed boundary. The balance

denominator should also be based on a manipulation-resistant measure that remains consistent as holdings change.

FAPR - Fresh Address Penalty Reset

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L217,466
Status	Unresolved

Description

Sell-window state is keyed only by the current seller. A holder can sell exactly 10% of a balance, transfer the remainder to a fresh address, and repeat with a new baseline. This allows most of a position to be liquidated at the base tier rather than the intended dump-penalty tier.

```
SellWindow storage window = sellWindows[seller];

uint256 newSold = soldInWindow + amount;
bool penalized =
    balanceAtStart > 0
    && newSold * PxtFeeModel.BPS
        > balanceAtStart * PxtFeeModel.PENALTY_THRESHOLD_BPS;
```

Recommendation

Design the sell quota so recently transferred balances do not obtain a fresh allowance.

IDUT - Incorrect Default Unlock Timestamp

Criticality	Minor / Informative
Location	PxtFeeModel.sol#L7 Pxt.sol#L75
Status	Unresolved

Description

`DEFAULT_SELL_UNLOCK_TIMESTAMP` is documented as 1 March 2027 at 00:00:00 UTC, but its value, `1_803_744_000`, corresponds to 27 February 2027 at 16:00:00 UTC. The intended timestamp is `1_803_859_200`, resulting in a 32-hour discrepancy.

When no environment override is supplied, the deployment script passes the incorrect default to the `Pxt` constructor. The constructor only verifies that the timestamp is in the future and stores it unchanged, causing sell restrictions and other timestamp-gated functionality to become available 32 hours earlier than documented.

```
// Production default: 2027-03-01 00:00:00 UTC.
uint256 constant DEFAULT_SELL_UNLOCK_TIMESTAMP = 1_803_744_000;

uint256 sellUnlock = vm.envOr(
    "SELL_UNLOCK_TIMESTAMP",
    DEFAULT_SELL_UNLOCK_TIMESTAMP
);

SELL_UNLOCK_TIMESTAMP = sellUnlockTimestamp;
```

Recommendation

Consider correcting the default timestamp to match the documented unlock date and adding deployment-time.

LFRU - LP Fees Remain Unallocated

Criticality	Minor / Informative
Location	PhoenixFeeCollector.sol#L62
Status	Unresolved

Description

When a configured pool produces nonzero LP fee growth, `collect` pulls those assets into the collector but does not assign `pulled` to any pending category.

`_payoutToken` pays only pre-existing donation, marketing, and burn obligations, so newly collected LP fees can remain unallocated.

```
pulled0 = IERC20(token0).balanceOf(address(this)) - before0;
pulled1 = IERC20(token1).balanceOf(address(this)) - before1;

_payoutToken(token0, pulled0);

if (bal == 0 || totalDue == 0) {
    emit FeesCollected(msg.sender, token, 0, 0, 0, 0, pulled);
    return;
}
```

Recommendation

Define an explicit allocation policy for LP fee growth and book or distribute each collected amount according to that policy. If the official pool is permanently fee-free, consider enforcing that configuration invariant.

MC - Missing Check

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L126 PhoenixBuyback.sol#L167,180,449 PhoenixBuybackMath.sol#L71
Status	Unresolved

Description

The protocol's privileged configuration functions do not fully validate their inputs before persisting them. `setOfficialPool()` only verifies that the pool contains PXT and does not ensure that `key.hooks` references the current hook. Similarly, the one-shot `configurePool()` does not verify that the key matches the official pool and configured hook, or that its currencies, tick spacing, and position ticks are valid. An incorrect value permanently binds these components to incompatible pools and can disable hook callbacks, liquidity management, fee collection, or buybacks.

Additionally, `setBuybackParams()` only rejects a zero recycle width. Values at or above `2^23` become negative when cast to `int24`, while smaller values can still overflow when multiplied by `tickSpacing`. Consequently, the assumption in `recycleTicks()` is not enforced, and buybacks may consistently revert after ownership has been renounced.

```
function setOfficialPool(PoolKey calldata key) external onlyOwner {
    if (officialPoolSet) revert OfficialPoolAlreadySet();
    if (!_poolContainsPxt(key)) revert InvalidPool();
    officialPoolId = key.toId();
    officialPoolSet = true;
}

function configurePool(PoolKey calldata key, int24 tickLower_, int24
tickUpper_, bytes32 salt_)
    external
    onlyOwner
{
    if (poolConfigured) revert PoolAlreadyConfigured();
    poolKey = key;
    tickLower = tickLower_;
    tickUpper = tickUpper_;
    positionSalt = salt_;
    poolConfigured = true;
}

if (recycleWidthSpacings_ == 0) revert InvalidRecycleWidth();

int24 width = int24(uint24(recycleWidthSpacings)) * spacing;
```

Recommendation

Consider validating that all configured pool keys consistently reference the expected currencies, hook, PoolManager pool, and usable tick parameters before committing one-shot state. The recycle-width limit should account for both the `int24` conversion and multiplication by the configured tick spacing; limiting it only to `type(int24).max` would not prevent multiplication overflow.

OPIIU - Official Pool Initialization Is Unrestricted

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L142
Status	Unresolved

Description

The hook does not enable the `beforeInitialize` permission and therefore cannot authorize the initializer or validate the pool's starting price. Once the deterministic hook address and `PoolKey` are known, any account can call the singleton `PoolManager` and initialize the official pool using an arbitrary valid `sqrtPriceX96`.

Uniswap v4 pool initialization is one-time, so the protocol's subsequent initialization transaction will revert. Recovery may require deploying a different hook or adopting the attacker-selected starting price, which could disrupt deployment and expose initial liquidity if seeding proceeds without additional validation.

```
function getHookPermissions() public pure override returns
(Hooks.Permissions memory) {
    return Hooks.Permissions({
        beforeInitialize: false,
        afterInitialize: false,
        beforeAddLiquidity: true,
        afterAddLiquidity: false,
        beforeRemoveLiquidity: false,
        afterRemoveLiquidity: false,
        beforeSwap: true,
        afterSwap: true,
        beforeDonate: false,
        afterDonate: false,
        beforeSwapReturnDelta: true,
        afterSwapReturnDelta: true,
        afterAddLiquidityReturnDelta: false,
        afterRemoveLiquidityReturnDelta: false
    });
}
```

Recommendation

Consider restricting initialization through an appropriate hook callback that validates the pool key, initializer, and intended starting price. Alternatively, deployment and initialization could be performed atomically so that no external account can initialize the pool first.

RGRP - Rotated Contracts Retain Privileges

Criticality	Minor / Informative
Location	Pxt.sol#L143,151 PhoenixV4ReturnDeltaHook.sol#L112
Status	Unresolved

Description

`Pxt.setFeeCollector()` and `setSellAttributor()` grant the replacement address contract-recipient approval but do not revoke the approval previously granted to the superseded address. Although the active protocol pointer changes, the old contract remains able to receive PXT unless governance performs a separate revocation.

Similarly, the hook marks every new fee collector as an authorized pre-unlock liquidity provider without clearing the previous collector's permission. Repeated rotations therefore accumulate historical privileged addresses, and replacing a compromised contract does not fully contain its permissions without additional transactions.

```
function setFeeCollector(address collector_) external onlyOwner {
    if (collector_ == address(0)) revert ZeroAddress();
    feeCollector = collector_;
    _setApprovedContractRecipient(collector_, true);
}

function setSellAttributor(ISellAttributor attributor_) external
onlyOwner {
    if (address(attributor_) == address(0)) revert ZeroAddress();
    sellAttributor = attributor_;
    _setApprovedContractRecipient(address(attributor_), true);
}

function setFeeCollector(PhoenixFeeCollector collector_) external
onlyOwner {
    if (address(collector_) == address(0)) revert ZeroAddress();
    feeCollector = collector_;
    liquidityProvider[address(collector_)] = true;
}
```


Recommendation

Consider making contract rotation update all permissions associated with both the previous and replacement addresses atomically. Where an address may legitimately retain an independent role, privilege ownership could be tracked explicitly so that obsolete permissions are removed without affecting separately authorized access.

STCBT - Seed Transfers Can Be Taxed

Criticality	Minor / Informative
Location	PhoenixBuyback.sol#L191 Pxt.sol#L274
Status	Unresolved

Description

`addLiquidity()` assumes that transferring `amount0Desired` and `amount1Desired` delivers those exact amounts to the collector. However, registering an address as `feeCollector` only exempts transfers where it is the sender. An inbound owner-to-collector PXT transfer remains subject to the 2.7% transfer fee unless the collector is separately assigned `FeeExempt` or `NoPenalty` status.

The function does not measure balance changes and proceeds with a caller-specified liquidity amount. An underfunded collector may therefore revert during PoolManager settlement, consume pre-existing PXT balances, or seed a different token composition than intended. Although the current bootstrap separately marks the collector as `FeeExempt`, this dependency is not enforced by the seeding path and may be missed during collector rotation or alternative deployments.

```

if (amount0Desired > 0) {
    IERC20(token0).safeTransferFrom(msg.sender, address(this),
amount0Desired);
}
if (amount1Desired > 0) {
    IERC20(token1).safeTransferFrom(msg.sender, address(this),
amount1Desired);
}

if (feeCollector != address(0) && from == feeCollector) {
    return PxtFeeModel.noFee(amount);
}

```

Recommendation

Consider deriving or validating the liquidity operation from the amounts actually received. The collector's required inbound-transfer exemption could also be established and verified as part of collector configuration rather than relying on a separate deployment step.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	src/core/Pxt.sol#L33,35,36
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
uint256 private immutable SELL_UNLOCK_TIMESTAMP
address public immutable DONATION_WALLET
address public immutable MARKETING_WALLET
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
V4Addresses	Library			
HookMiner	Library			
	find	Internal		
	computeAddress	Internal		
PhoenixV4ReturnDeltaHook	Implementation	BaseHook, Ownable, PxtFeeEvents, ISellAttributor		
		Public	✓	BaseHook Ownable
	setFeeCollector	External	✓	onlyOwner
	setLiquidityProvider	External	✓	onlyOwner
	setOfficialPool	External	✓	onlyOwner
	sellProtectionCleared	External		-
	antiBotSeller	External		-
	getHookPermissions	Public		-
	flags	Public		-
	_beforeAddLiquidity	Internal		
	_beforeSwap	Internal	✓	

	_afterSwap	Internal	✓	
	attributeSell	External	✓	-
	finalizeOrphanedSell	Public	✓	-
	_beforeSell	Internal	✓	
	_beforeBuy	Internal	✓	
	_afterSell	Internal	✓	
	_afterBuy	Internal	✓	
	_accrueUsdc	Internal	✓	
	_fairSellFeeBps	Internal		
	_applySellWindow	Internal	✓	
	_enforceOfficialPool	Internal		
	_poolContainsPxt	Internal		
	_pxtIsCurrency0	Internal		
	_quoteCurrency	Internal		
	_pxtSwapAmount	Internal		
	_quoteSwapAmount	Internal		
	_classifySwap	Internal		
	_pxtIsSpecified	Internal		
	_recordPendingSkim	Internal	✓	
	_clearPendingSkim	Internal	✓	
	_tstore	Internal	✓	
	_tload	Internal		

ISellAttributor	Interface			
	attributeSell	External	✓	-
PhoenixFeeCollector	Implementation	PhoenixBuyback		
		Public	✓	PhoenixBuyback
	setHook	External	✓	onlyOwner
	receiveAccruedFees	External	✓	onlyHook
	collect	External	✓	nonReentrant
	_payoutToken	Internal	✓	
	_reconcileWalletBurnPending	Internal	✓	
PhoenixBuybackMath	Library			
	pxtForQuote	Internal		
	enforceMinOut	Internal		
	sqrtPriceLimitAfterSlippage	Internal		
	floorToSpacing	Internal		
	recycleTicks	Internal		
	isPxtOnlyBand	Internal		
PhoenixBuyback	Implementation	IUnlockCallback, Ownable, ReentrancyGuard		
	_onlyHook	Internal		

		Public	✓	Ownable
	configurePool	External	✓	onlyOwner
	setBuybackParams	External	✓	onlyOwner
	addLiquidity	External	✓	onlyOwner nonReentrant
	recycleBandCount	External		-
	pruneEmptyRecycleBands	External	✓	nonReentrant
	_pullLpFees	Internal	✓	
	_collectPositionFees	Internal	✓	
	_requireTradingOpen	Internal		
	executeBuyback	External	✓	nonReentrant
	_executeBuybackCash	Internal	✓	
	pending	External		-
	quoteBuyback	External		-
	unlockCallback	External	✓	-
	_burnToken	Internal	✓	
	_addRecycleLiquidity	Internal	✓	
	_selectPxtOnlyRecycleBand	Internal		
	_registerRecycleBand	Internal	✓	
	_pruneEmptyRecycleBands	Internal	✓	
	_removeRecycleBandAt	Internal	✓	
IPxtSellControls	Interface			
	sellUnlockTimestamp	External		-

	antiBotSeller	External		-
	sellProtectionCleared	External		-
PxtSellAccess	Library			
	enforceSellUnlocked	Internal		
	enforceSellAccess	Internal		
	enforceTradingOpen	Internal		
PxtFeeEvents	Implementation			
PxtFeeModel	Library			
	noFee	Internal		
	splitBuy	Internal		
	splitSellBurn	Internal		
	splitSellUsdc	Internal		
Pxt	Implementation	ERC20, ERC20Permit , Ownable, AccessContr ol, PxtFeeEvent s		
		Public	✓	ERC20 ERC20Permit Ownable
	decimals	Public		-
	sellUnlockTimestamp	Public		-
	setPoolManager	External	✓	onlyOwner

	setAntiBotSeller	External	✓	onlyOwner
	clearSellProtection	External	✓	-
	setFeeCollector	External	✓	onlyOwner
	setSellAttributor	External	✓	onlyOwner
	setApprovedContractRecipient	External	✓	onlyRole
	setWalletStatus	External	✓	onlyOwner
	quoteTransfer	External		-
	burnBalance	External	✓	-
	_setApprovedContractRecipient	Internal	✓	
	_isProtectedRecipient	Internal		
	_isWallet	Internal		
	_enforceContractRecipient	Internal		
	_enforceSellAccess	Internal		
	_update	Internal	✓	
	_onDexSell	Internal	✓	
	_quoteTransfer	Internal		

Summary

Phoenix Token implements a fixed-supply ERC-20 ecosystem with Uniswap v4 trading fees, sell protections, dump penalties, USDC fee collection, and automated PXT buybacks. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io